Week 5: Markov models and Tensor Backpropagation

https://mlvu.github.io

March 4, 2025

1 Markov model classification

This exercise is a follow-up to the naive Bayes classifier of the 3rd week. We recommend briefly reviewing that one. This exercise will be much easier if you do so.

In the lecture, the Bayes sequence classification was explained. The result of training is the frequency of words in different classes. Assume that we train an email spam classifier, and Table 1 shows the frequency of the words in spam and ham emails with a **first-order Markov model**. In total, the number of words in spam and ham emails is 20,000 and 100,000, respectively.

Table 1: Frequency of the words in spam and ham emails.

bigrams & unigrams	frequency	
	spam	ham
claim	2000	1000
prize	5000	1000
you	2000	2000
won	2000	1000
claim prize	300	200
prize you	500	100
you won	800	200

We receive the following four-word email.

email: claim prize you won

To classify this email, we are going to use the *chain rule of probability* to to simplify the conditional probability $\mathrm{p}(\text{claim}, \text{prize}, \text{you}, \text{won} \mid \text{spam})$.

question 1: If we are ultimately interested in the probability $p(\text{spam} \mid \text{claim}, \text{prize}, \text{you}, \text{won})$, then how do we end up with the probability above?

This is because we apply Bayes' rule to reverse the conditional. If we represent the content of the email as E, this reads

$$p(\text{spam} \mid E) = \frac{p(E \mid \text{spam})\, p(\text{spam})}{p(E)}$$

Since the chain rule doesn't apply to things in the conditional, we need do this reversal first, and then work out the factors on the right as necessary.

question 2: Take the value $p(\text{claim}, \text{prize}, \text{you}, \text{won} \mid \text{spam})$ and apply the chain rule first (so that every word becomes conditioned on the words before it). Then, apply the Markov assumption, with order 1.

Applying the chain rule, step by step:

$$\begin{aligned}
p(\text{claim}, \text{prize}, \text{you}, \text{won} \mid \text{spam}) &= p(\text{prize}, \text{you}, \text{won} \mid \text{claim}, \text{spam}) \cdot p(\text{claim} \mid \text{spam}) \\
&= p(\text{you}, \text{won} \mid \text{claim}, \text{prize}, \text{spam}) \cdot \\
&\quad\; p(\text{prize} \mid \text{claim}, \text{spam}) \cdot \\
&\quad\; p(\text{claim} \mid \text{spam}) \\
&= p(\text{won} \mid \text{claim}, \text{prize}, \text{you}, \text{spam}) \cdot \\
&\quad\; p(\text{you} \mid \text{claim}, \text{prize}, \text{spam}) \cdot \\
&\quad\; p(\text{prize} \mid \text{claim}, \text{spam}) \\
&\quad\; p(\text{claim} \mid \text{spam})
\end{aligned}$$

The result is that we have a product of probabilities of one word. To make these easy to estimate, we apply the (first order) *Markov assumption*, that words are dependent only on the one word directly before it:

$$\begin{aligned}
p(\text{claim}, \text{prize}, \text{you}, \text{won} \mid \text{spam}) = {} & p(\text{won} \mid \text{you}, \text{spam}) \cdot \\
& p(\text{you} \mid \text{prize}, \text{spam}) \cdot \\
& p(\text{prize} \mid \text{claim}, \text{spam}) \\
& p(\text{claim} \mid \text{spam})
\end{aligned}$$

With this, we have a set of probabilities that we can easily estimate from the data.

question 3: To fill in the rest of Bayes' rule, we also need the prior probability $p(\text{spam})$, that an email is spam. Give two ways we might estimate this in real life? The first is from the data. We just divide the number of spam emails by the total number of emails. The second is to use knowledge of the real world. Maybe the dataset is artificially balanced. In that case, we could do a survey or look up statistics about how likely emails in general are to be spam.

Assume we have two priors. Prior 1 says that emails are three times as likely to be ham as spam. Prior 2 says that the probability of a spam email is 1/40.

question 4: Under prior 1, is the email above more likely to be spam or ham?

The first thing to note is that if we just want a classification, not a probability, we don't need to compute the whole of Bayes' rule. Let E represent the whole email, then the probabilities we want to compare are

$$p(\text{spam} \mid E) = \frac{p(E \mid \text{spam})p(\text{spam})}{p(E)} \qquad p(\text{ham} \mid E) = \frac{p(E \mid \text{ham})p(\text{ham})}{p(E)}$$

The denominator $p(E)$ is the same in both cases, so if we want to know whether we get a spam classification, we can just ask if $p(E \mid \text{spam})p(\text{spam})$ is higher than $p(E \mid \text{ham})p(\text{ham})$.

As we will see later, once we've computed these factors, it's then relatively easy to compute the class probabilities as well.

For $p(E \mid \text{spam})$, we use the chain rule as above. For each resulting factor, we can estimate a probability like $p(\text{won} \mid \text{you}, \text{spam})$ as the number of times the word "you" was followed by "won", divided by the total number of times we saw "you" (in the spam part of the data).

$$\begin{aligned} p(\text{claim}, \text{prize}, \text{you}, \text{won} \mid \text{spam}) = {} & p(\text{won} \mid \text{you}, \text{spam}) \cdot && \frac{800}{2\,000} = \frac{2}{5} \\ & p(\text{you} \mid \text{prize}, \text{spam}) \cdot && \frac{500}{5\,000} = \frac{1}{10} \\ & p(\text{prize} \mid \text{claim}, \text{spam}) \cdot && \frac{300}{2\,000} = \frac{3}{20} \\ & p(\text{claim} \mid \text{spam}) && \frac{2\,000}{20\,000} = \frac{1}{10} \end{aligned}$$

Note that for the last factor, we need to estimate the probability of the word "claim" without conditioning on the preceding word. We do this just

by dividing the total number of times we saw it in all spam emails divided by the the total number of words in the whole spam part of the data.

We could punch all this into a calculator, but we recommend doing it by hand as follows:

$$\mathrm{p}(\mathsf{E} \mid \text{spam}) = \frac{2}{5}\frac{1}{10}\frac{3}{20}\frac{1}{10} = \frac{2 \cdot 3}{5 \cdot 2 \cdot 10 \cdot 10 \cdot 10} = \frac{3}{5} \cdot 10^{-3} \,.$$

Next up, the computation for ham:

$$\begin{aligned}
\mathrm{p}(\text{claim}, \text{prize}, \text{you}, \text{won} \mid \text{ham}) = \mathrm{p}(\text{won} \mid \text{you}, \text{ham}) \cdot & \quad \frac{200}{2\,000} = \frac{1}{10} \\
\mathrm{p}(\text{you} \mid \text{prize}, \text{ham}) \cdot & \quad \frac{100}{1\,000} = \frac{1}{10} \\
\mathrm{p}(\text{prize} \mid \text{claim}, \text{ham}) \cdot & \quad \frac{200}{1\,000} = \frac{1}{5} \\
\mathrm{p}(\text{claim} \mid \text{ham}) & \quad \frac{1\,000}{100\,000} = \frac{1}{100} \,.
\end{aligned}$$

Which gives us

$$\mathrm{p}(\mathsf{E} \mid \text{ham}) = \frac{1}{10}\frac{1}{10}\frac{1}{5}\frac{1}{100} = \frac{1}{5 \cdot 10 \cdot 10 \cdot 100} = \frac{1}{5} \cdot 10^{-4} \,.$$

Now, we must be careful here: the value we've just computed is much lower for ham than for spam, but we're not finished yet, we still need to multiply by the prior. Prior 1 tells us that $\mathrm{p}(\text{spam}) = \frac{1}{4}$.

This gives us

$$\begin{aligned}
\mathrm{p}(\text{spam} \mid \mathsf{E}) \propto \mathrm{p}(\mathsf{E} \mid \text{spam})\,\mathrm{p}(\text{spam}) = \frac{3}{5} \cdot 10^{-3} \cdot \frac{1}{4} = \frac{3}{20} \cdot 10^{-3} \\
\mathrm{p}(\text{ham} \mid \mathsf{E}) \propto \mathrm{p}(\mathsf{E} \mid \text{ham})\,\mathrm{p}(\text{ham}) = \frac{1}{5} \cdot 10^{-4} \cdot \frac{3}{4} = \frac{3}{20} \cdot 10^{-4}
\end{aligned}$$

In short, the prior doesn't affect the result sufficiently to change the outcome: spam is still more likely than ham.

question 5: Under prior 2, is the email above more likely to be spam or ham?

We can re-use most of our work from the previous answer.

Prior 2 tells us that $\mathrm{p}(\text{spam}) = \frac{1}{40}$.

This gives us

$$\mathrm{p}(\text{spam} \mid \mathsf{E}) \propto \mathrm{p}(\mathsf{E} \mid \text{spam})\,\mathrm{p}(\text{spam}) = \frac{3}{5} \cdot 10^{-3} \cdot \frac{1}{40} = \frac{3}{200} \cdot 10^{-3} = \frac{30}{200} \cdot 10^{-4}$$
$$\mathrm{p}(\text{ham} \mid \mathsf{E}) \propto \mathrm{p}(\mathsf{E} \mid \text{ham})\,\mathrm{p}(\text{ham}) = \frac{1}{5} \cdot 10^{-4} \cdot \frac{39}{40} = \frac{39}{200} \cdot 10^{-4}$$

There's no standard way to get the probabilities into a form like this that makes them easy to compare. It takes a little trial and error.

That is, under this prior, ham is more likely than spam. The point is that even though the content of the email points towards spam, in the world suggested by this prior we see so few spam emails, that we need stronger evidence to convince us to judge this email to be spam.

question 6: Compute the full class probabilities under prior 2. The simplest way to get from the unnormalized probabilities of the previous question to the complete class probabilities is to rewrite Bayes' rule as follows:

$$\begin{aligned}\mathrm{p}(\text{spam} \mid \mathsf{E}) &= \frac{\mathrm{p}(\mathsf{E} \mid \text{spam})\mathrm{p}(\text{spam})}{\mathrm{p}(\mathsf{E})} \\ &= \frac{\mathrm{p}(\mathsf{E} \mid \text{spam})\mathrm{p}(\text{spam})}{\mathrm{p}(\mathsf{E} \mid \text{spam})\mathrm{p}(\text{spam}) + \mathrm{p}(\mathsf{E} \mid \text{ham})\mathrm{p}(\text{ham})}\end{aligned}$$

This looks a bit more complicated, but it shows more intuitively what Bayes rule means. *See this slide for an explanation.*

We now see a payoff for working out the probabilities by hand rather than using a calculator. If we fill them in, we see that a lot of factors cancel out:

$$\mathrm{p}(\text{spam} \mid \mathsf{E}) = \frac{\frac{30}{200} \cdot 10^{-4}}{\frac{30}{200} \cdot 10^{-4} + \frac{39}{200} \cdot 10^{-4}} = \frac{30}{30+39} = \frac{30}{69}\,.$$

Since the probabilities of the emails being ham and spam should add to 1, we know that $\mathrm{p}(\text{ham} \mid \mathsf{E})$ should be $39/69$. By filling the in the probabilities, we see how Bayes' rule ensures this:

$$\mathrm{p}(\text{ham} \mid \mathsf{E}) = \frac{\frac{39}{200} \cdot 10^{-4}}{\frac{30}{200} \cdot 10^{-4} + \frac{39}{200} \cdot 10^{-4}} = \frac{39}{30+39} = \frac{39}{69}\,.$$

In short, the unnormalized probabilities we produced in the previous two questions should sum to one if we make them into proper propbabilities,

so we can just divide them by the total over all the classes. IT turns out that this is exactly what Bayes' rule does.

One way of expressing the certainty of the classification ham is by computing the probability ratio of ham over spam, or the *odds* of the class ham (according to the classifier).

question 7: Using prior 1, compute the *probability ratio* of spam over ham. What does it tell you?

The ratio we're interested in is

$$\frac{p(\text{spam} \mid E)}{p(\text{ham} \mid E)} .$$

We could work out the probabilities as we did in the previous question (but with prior 1), but note that if we fill in de definition of Bayes' rule, the denominators cancel out:

$$\frac{p(\text{spam} \mid E)}{p(\text{ham} \mid E)} = \frac{p(E \mid \text{spam})p(\text{spam})\frac{1}{p(E)}}{p(E \mid \text{ham})p(\text{ham})\frac{1}{p(E)}} .$$

Therefore, all we need to work out is the ratio between the unnormalized probabilities. For prior 1, this gives us:

$$\frac{p(\text{spam} \mid E)}{p(\text{ham} \mid E)} = \frac{\frac{3}{20} \cdot 10^{-3}}{\frac{3}{20} \cdot 10^{-4}} = \frac{10}{1} = 10 .$$

Note that the probability ratio can easily be larger than 1. In fact, if it is larger than one, it tells us that spam is the more likely class. The closer to 1, however, the less sure the classifier is of its judgment.

Some people call this ratio *the odds of spam* or *the odds in favor of spam*. Odds of 10/1 in favor of spam mean that on average, in every 11 emails, 10 will be ham. This means that if you bet 1 euro that the next email will be ham I can safely pay out 10 euros if the bet comes true, without making a loss on average.

1.1 Extra practice

We are given a dataset of email, labeled spam and ham, where the number of spam emails is 1% of the total. The total number of words in the spam and ham datasets is is 50 000 and 60 000, respectively.

On this dataset, we want to train a *first-order Markov model* as a basis for a Bayes classifier to detect spam.

The following table shows the frequency of several bigrams and unigrams in the dataset.

uni- and bigrams	frequency	
	spam	ham
congratulations	4 000	1 000
you	5 000	5 000
won	8 000	3 000
bitcoin	2 000	2 000
lottery	3 000	500
congratulations you	1 000	200
you won	1 000	500
won bitcoin	1 000	100
bitcoin lottery	800	100
won lottery	500	100
lottery congratulations	40	50

Consider two emails with the following contents:

email 1:	congratulations you won bitcoin lottery
email 2:	you won bitcoin lottery congratulations

Use the data to estimate the prior probabilities of ham and spam.

question 8: How are the two emails classified?

A Both are spam.
B The first is spam, the second is ham. ✓
C The first is ham, the second is spam.
D Both are ham.

question 9: What is the probability that the first email is spam?

A $\frac{11}{51}$ This is p(ham) **B** $\frac{40}{51}$ ✓ **C** $\frac{99}{199}$ **D** $\frac{100}{199}$

question 10: What is the *probability ratio* of spam over ham for the second email?

A $\frac{32}{330}$ ✓ **B** $\frac{330}{32}$ **C** $\frac{37}{99}$ **D** $\frac{99}{37}$

2 Backpropagation Revisited

2.1 The multivariate chain rule

If we want to do backpropagation for a computation graph where an output variable depend in an input variable through multiple intermediate values (the graph contains a diamond), we require the *multivariate chain rule*. If you don't quite remember how it goes, re-read slides 24–29 fo the deep learning lecture before trying these questions.

We will use the backpropagation algorithm to find the derivative of the function

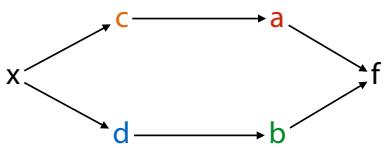
$$f(x) = \sin(x^2)\cos(x^3)$$

with respect to x.

question 11: First, we break the function up into modules. Fill in the blanks.

$$\begin{aligned} f &= ab \\ a &= \sin(c) \\ b &= \cos(d) \\ c &= x^2 \\ d &= x^3 \end{aligned}$$

question 12: Draw the computation graph with nodes x, c, d, a, b, f



question 13: Work out the *local* derivatives. Which is the correct expression for the gradient $\partial f/\partial x$?

$$
\begin{aligned}
\frac{\partial f}{\partial x} &= \frac{\partial f}{\partial a}\frac{\partial a}{\partial x} + \frac{\partial f}{\partial b}\frac{\partial b}{\partial x} && \text{multivariate chain rule} \\
&= \frac{\partial f}{\partial a}\frac{\partial a}{\partial c}\frac{\partial c}{\partial x} + \frac{\partial f}{\partial b}\frac{\partial b}{\partial x} && \text{chain rule for } c\text{: } \frac{a}{x} = \frac{\partial a}{\partial c}\frac{\partial c}{\partial x} \\
&= \frac{\partial f}{\partial a}\frac{\partial a}{\partial c}\frac{\partial c}{\partial x} + \frac{\partial f}{\partial b}\frac{\partial b}{\partial d}\frac{d}{x} && \text{chain rule for } d \\
&= b \cdot \cos(c) \cdot 2x - a \cdot \sin(d) \cdot 3x^2 && \text{work out all local derivatives}
\end{aligned}
$$

This is a common exam question so make sure to practice if you're not sure. You can easily create new questions for yourself by coming up with any function $f(x)$ with a diamond shape in the computation (just make sure that x is used twice).

2.2 Tensor Backpropagation

In scalar backpropagtion, we need to work out only the local derivatives. Once we have these, we can multiply them in any order to get the global derivative. If we want to apply backpropagation to tensors, things are not so easy.

question 14: Why not? What is it about the local derivatives in tensor backpropagation that makes it difficult to apply in this way?

In tensor backpropagation, the local derivatives are often, for instance, the derivative of a vector function over matrix parameters, like the local derivative $\partial \mathbf{k}/\partial W$ shown in the slides. In this case, the collection of all scalar derivatives (every output with every input) is best represented as a 3 tensor. This would take way too much memory, and there's then no natural way to multiply this local derivative with the others to compute the global derivative.

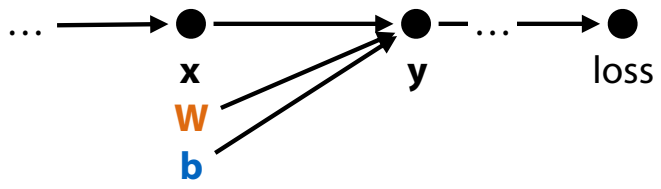
The solution is not to compute the local derivatives explicitly, but only to compute the gradients on the inputs over the loss, given the gradients of the outputs over the loss.

We will practice this for a simple feedforward layer (without activation). Consider a module f that computes:[1]

$$\mathbf{y} = f(\mathbf{x}, W, \mathbf{b}) = W\mathbf{x} + \mathbf{b}$$

[1] Normally, we would consider $\mathbf{x}$ the input, and W and $\mathbf{b}$ the parameters, but for our AD engine, the distinction does not matter: everything going in to the computation is an input, and we may need the gradient over all three.

Assume that this module is part of a much larger network. Its output $\mathbf{y}$ is fed as input to another module, which produces a new output that is fed to another module and so on. At the end, a single scalar loss L is produced.



Ultimately, we want to work out the derivative of this loss, with respect to our inputs:

$$\mathbf{x}^{\nabla}, \quad W^{\nabla}, \quad \mathbf{b}^{\nabla}$$

Where the notation W^{∇} represents a matrix for the scalar derivatives of the loss with respect to elements of W. For instance, element (2, 3) of matrix W^{∇} is the scalar derivative $\partial l / \partial W_{23}$.

We'll start with the bias term $\mathbf{b}$. Since matrix/vector calculus can get complicated, and doesn't translate to tensors of higher rank, we'll develop everything in terms of scalar calculus. Once we've taken the derivatives we'll transform everything to matrix operations to make the implementation efficient.

We're interested in $\frac{\partial L}{\mathbf{y}} \frac{\partial \mathbf{y}}{\partial b_j}$, but we need matrix/vector calculus to do that. Instead we will consider $(f(\mathbf{b}))$ as a *scalar* computation, which has multiple intermediate values $y_1, y_2 \ldots, y_k$. By the multivariate chain rule, we can just take the derivative over each path (through each value y_i), and sum them:

$$\frac{\partial l}{\partial b_j} = \sum_i \frac{\partial l}{\partial y_i} \frac{\partial y_i}{\partial b_j} = \sum_i y_i^{\nabla} \frac{\partial y_i}{\partial b_j}$$

We will assume that $\mathbf{y}^{\nabla}$ is given to us by the automatic differentiation (AD) engine as a vector. All we need to work out is a simple matrix operation that computes $\partial l / \partial b_j$ for all j and lays the results out in the same shape as $\mathbf{b}$. We'll start by working out the scalar derivative.

question 15: Fill in the gaps

$$\begin{aligned}\frac{\partial l}{\partial b_j} &= \sum_i \frac{\partial l}{\partial y_i}\frac{\partial y_i}{\partial b_j} = \sum_i y_i^{\nabla}\frac{\partial y_i}{\partial b_j} \\ &= \sum_i y_i^{\nabla}\frac{\partial [\boldsymbol{W}\boldsymbol{x}+\boldsymbol{b}]_i}{\partial b_j} = \sum_i y_i^{\nabla}\frac{\partial [\boldsymbol{W}\boldsymbol{x}]_i + b_i}{\partial b_j} \\ &= \sum_i y_i^{\nabla}\frac{\partial b_i}{\partial b_j} = y_j^{\nabla}\end{aligned}$$

question 16: The `backward()` function for `f` if given $\boldsymbol{y}^{\nabla}$. What should it return as the gradient for $\boldsymbol{b}$?

We should return a vector $\boldsymbol{b}'$ with the same shape as $\boldsymbol{b}$, where $b_j' = \partial l/\partial b_j$. Since y_j^{∇} is the gradient for b_j, we can just return the vector $\boldsymbol{y}^{\nabla}$.

We'll do the same for the gradient over $\boldsymbol{x}$.

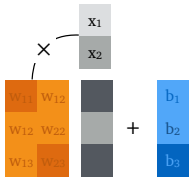
question 17: Work out the scalar derivative $\partial l/\partial x_j$ using the multivariate chain rule (again taking $y_i^{\nabla} = \partial l/\partial y_i$ as given).

$$\begin{aligned}\frac{\partial l}{\partial x_j} &= \sum_i \frac{\partial l}{\partial y_i}\frac{\partial y_i}{\partial x_j} = \sum_i y_i^{\nabla}\frac{\partial y_i}{\partial x_j} = \sum_i y_i^{\nabla}\frac{\partial [\boldsymbol{W}\boldsymbol{x}+\boldsymbol{b}]_i}{\partial x_j} \\ &= \sum_i y_i^{\nabla}\frac{\partial [\boldsymbol{W}\boldsymbol{x}]_i + b_i}{\partial x_j} = \sum_i y_i^{\nabla}\frac{\partial [\boldsymbol{W}\boldsymbol{x}]_i}{x_j} + \frac{\partial b_i}{\partial x_j} = \sum_i y_i^{\nabla}\frac{\partial \boldsymbol{W}_{i\cdot}\boldsymbol{x}}{x_j} \\ &= \sum_i y_i^{\nabla}\frac{\partial \sum_k \boldsymbol{W}_{ik}x_k}{\partial x_j} = \sum_{i,k} y_i^{\nabla}\frac{\partial \boldsymbol{W}_{ik}x_k}{\partial x_j} = \sum_i y_i^{\nabla}\frac{\partial \boldsymbol{W}_{ij}x_j}{\partial x_j} \\ &= \sum_i y_i^{\nabla}\boldsymbol{W}_{ij}\end{aligned}$$

question 18: What should the `backward()` function for `f` return as the gradient for $\boldsymbol{x}$?

We are looking for a function that returns a vector $\boldsymbol{x}'$ where $x_j' = \sum_i y_i^{\nabla}\boldsymbol{W}_{ij}$. In other words, x_j' should be the dot product between $\boldsymbol{y}^{\nabla}$ and and the j-th column of $\boldsymbol{W}$. We get this if we compute $\boldsymbol{x}' = \boldsymbol{y}^{\nabla\mathsf{T}}\boldsymbol{W}$.

forward



backward for **x**